A Comparison of Heuristic and Metaheuristic Methods for solving the Traveling Salesman Problem

1st Daniel Schafhäutle
Computational and Data Science
FHGR
Chur, Switzerland
daniel.schafhautle@stud.fhgr.ch

2nd Joshua Kohler
Computational and Data Science
FHGR
Chur, Switzerland
joshua.kohler@stud.fhgr.ch

3rd Gian-Luca Lanfranchi
Computational and Data Science
FHGR
Chur, Switzerland
gianluca.lanfranchi@stud.fhgr.ch

***Abstract*—The traveling salesman problem (TSP) is an NP hard combinatorial optimization problem, that asks for the shortest possible route that visits a set of cities and returns to the starting city. Solving TSPs exactly using brute-force methods quickly becomes computationally infeasible as the number of cities increases. However, there exists a number of heuristic and metaheuristic algorithms, that can find reasonably good solutions in a much shorter time. This paper presents a comparison of five such methods, namely: Simulated Annealing, Tabu Search, 2-Opt, Iterative Local Search and a Genetic Algorithm. Algorithms are evaluated using benchmark TSP instances of sizes n = [10, 25, 50, 100, 200, 500] with k = 30 instances each. We preform this experiment with two types of instances: one with uniformly sampled cities and one with clustered cities. They are then compared based on the solution quality measured as the gap to the optimal solution obtained by the Concorde TSP solver, and the time required to find the solution. The goal of this paper is to compare the strengths and weaknesses of these algorithms, that are the basis of many state of the art methods for solving TSPs and provide a starting point for further research into this area.**

I. Introduction

The traveling sailsman problem is a solved

Problem Definition: Formally define the Traveling Salesman Problem (TSP). Use standard mathematical notation here: given a set of nodes (cities) V and edge weights (distances) d_{ij}, find a permutation of cities that minimizes the total tour length:

$$\min \sum_{i=1}^{n-1} d_{\pi(i),\pi(i+1)} + d_{\pi(n),\pi(1)}$$

Complexity: Explain why it matters (NP-hard, combinatorial explosion where brute force requires $O(n!)$ operations).

The Heuristic Objective: Explain that because exact solutions scale terribly, we trade a guarantee of absolute optimality for speed, aiming for "reasonably good" solutions

II. Related Work

Mention that exact solvers like Concorde use cutting-plane methods and branch-and-bound to find the true mathematical optimum.

Briefly reference classic literature (like the Glover paper mentioned in your README) regarding how local search and metaheuristics revolutionized combinatorial optimization.

III. Materials and Methods

A. 2-Opt

B. Simulated Annealing

C. Tabu Search

D. Iterative Local Search

E. Genetic Algorithm

IV. Experiments and Results

Detail exactly how you conducted the experiment so someone else could replicate it

A. Setup

1) Dataset Generation:

2) Benchmark:

3) Hardware / Environment:

B. Results

- Performance Metrics
- Plots

V. Discussion

VI. Conclusion

Summarize core finding.
suggest future directions

- Other algorithms. Hyperparameter tuning.

Wollt ihr die Referenzen manuell oder mit z.B. Zotero machen?

References

[1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.

VII. The First Appendix